Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій
Кафедра системного проектування

Звіт

про виконання лабораторної роботи №5

“Обробка помилок, файли та серіалізація”

Виконав:
студент 3 курсу
групи ФЕП-33с
Качмар Денис

Викладач:
Проф. Свелеба Сергій Андрійович

Львів - 2026

ХІД РОБОТИ

Тема: Обробка помилок, файли та серіалізація
Мета: Сформувати практичні навички побудови надійних, відмовостійких Python-програм, які коректно управляють винятковими ситуаціями, файловими ресурсами, конфігураціями та структурованими форматами даних. Опанувати механізм exceptions, конструкції `try-except-else-finally`, створення ієрархії custom exceptions, exception chaining (`raise ... from ...`), підходи EAFP та fail-fast, протокол context managers і модуль `contextlib`, сучасну роботу зі шляхами через `pathlib.Path`, роботу з форматами CSV, JSON, JSON Lines, YAML, потоковий streaming I/O без завантаження всього набору в пам'ять, атомарний запис файлів (atomic replace), структуроване логування (`logging`) без витоку конфіденційної інформації та підтримку двох стратегій валідації (strict та tolerant mode).
Варіант: 5 (Каталог автомобілів / Car Catalog)

Постановка задачі: Створити надійний інфраструктурний модуль імпорту, експорту, валідації та конфігурації для системи обліку каталогу автомобілів (Варіант 5).
Поля сутності: `vehicle_id`, `brand`, `model`, `year`, `price`, `mileage`.
Вимоги та функціонал: 1. Використання об'єктів `pathlib.Path` для платформонезалежної роботи з файловою системою. 2. Потокове читання вхідних даних (streaming CSV import через `csv.DictReader` без повної матеріалізації в списки). 3. Багаторівнева валідація на межі системи (system boundary): `vehicle_id > 0`, непорожні `brand` та `model`, коректний рік випуску (1886–2026), `price >= 0`, `mileage >= 0`. 4. Власна доменна ієрархія винятків (`ApplicationError` -> `DataError` -> `RecordValidationError`, `DataImportError`, `DataExportError`, `ConfigurationError`). 5. Збереження першопричин помилок через явний ланцюжок винятків (`raise RecordValidationError(...) from error`). 6. Зовнішня конфігурація у форматі YAML (`config.yaml`) із підтримкою діапазонів фільтрації років та цін, шляхів вводу/виводу, прапорця `skip_invalid`, налаштувань логування та підтримки перевизначення через змінні середовища (Environment-variable override). 7. Раннє виявлення помилок конфігурації (Fail-fast validation). 8. Структуроване логування через модуль `logging` із записом у консоль та ротаційний/текстовий лог-файл, усунення конфіденційних даних із журналу. 9. Підтримка двох режимів обробки: - **Strict mode** (`skip_invalid: false`): аварійне завершення на першому некоректному записі з фіксацією помилки; - **Tolerant mode** (`skip_invalid: true`): фіксація попередження у log із зазначенням номера рядка і поля, збереження некоректного запису в окремий error CSV, інкремент лічильника invalid та продовження потоку. 10. Атомарний експорт у JSON (streaming JSON array) та JSON Lines із використанням тимчасового файлу (`.tmp`), що запобігає пошкодженню цільових файлів у разі аварійного збою програми. 11. Розробка власних context managers: `@contextmanager def logged_operation` для моніторингу часу та помилок транзакцій і `atomic_file_writer` для безпечних операцій запису. 12.

Реалізація завдань підвищеної складності: протоколи **`Importer`** та **`Exporter`**, єдиний інтерфейс обробки для CSV, JSON, JSONL та YAML, автовибір формату за розширенням, окремий експорт журналу невалідних записів у CSV, підтримка резервного копіювання існуючих файлів (**`backup: true`**).

---

1. Схема вхідних даних (Input Data Schema)

Згідно з варіантом 5, кожен запис каталогу автомобілів містить 6 обов'язкових атрибутів:

Поле	Тип	Обмеження валідації	Опис
`vehicle_id`	**`int`**	**`vehicle_id > 0`**	Унікальний цілочисельний ідентифікатор транспортного засобу (допускається назва **`id`** або **`vehicle_id`**)
`brand`	**`str`**	Непорожній рядок (**`strip() != ""`**)	Марка автовиробника (Audi, BMW, Toyota, Ford тощо)
`model`	**`str`**	Непорожній рядок (**`strip() != ""`**)	Комерційна назва моделі авто (A4, X5, Camry тощо)
`year`	**`int`**	**`1886 <= year <= 2026`**	Рік випуску автомобіля (від створення першого авто Карла Бенца до поточного року)
`price`	**`float`**	**`price >= 0.0`**	Ринкова вартість автомобіля у доларах США
`mileage`	**`float`**	**`mileage >= 0.0`**	Експлуатаційний пробіг автомобіля в кілометрах

---

2. Приклад вхідного CSV-файлу (data/cars_sample.csv)

Для демонстрації роботи конвеєра, валідації та обох стратегій обробки (strict та tolerant) сформовано змішаний набір даних, що містить як коректні, так і типові пошкоджені записи:

```
vehicle_id,brand,model,year,price,mileage
1,Audi,A4,2018,22500.00,65000.0
2,BMW,320i,2020,31000.00,42000.0
3,Toyota,Corolla,2015,12000.00,110000.0
```

```
4,,Golf,2017,14500.00,78000.0
5,Ford,Focus,1800,8500.00,150000.0
abc,Mercedes-Benz,C200,2019,27000.00,55000.0
7,Skoda,Octavia,2021,-18000.00,30000.0
8,Honda,Civic,2022,24000.00,25000.0
9,Mazda,6,2016,13500.00,-45000.0
10,Volvo,XC60,2023,48000.00,15000.0
```

Характеристика дефектів у тестових записах: - **Рядок 5 (id=4):** порожнє поле **`brand`** (порушення непорожнього рядка); - **Рядок 6 (id=5):** рік **`1800`** виходить за межі діапазону **`1886-2026`**; - **Рядок 7 (id='abc'):** нечисловий ідентифікатор **`abc`** (викликає **`ValueError`** при парсингу в **`int`**); - **Рядок 8 (id=7):** від'ємна вартість **`-18000.00`**; - **Рядок 10 (id=9):** від'ємний пробіг **`-45000.0`**.

---

3. Структура проєкту

Проєкт організовано відповідно до сучасних стандартів Python-розробки:

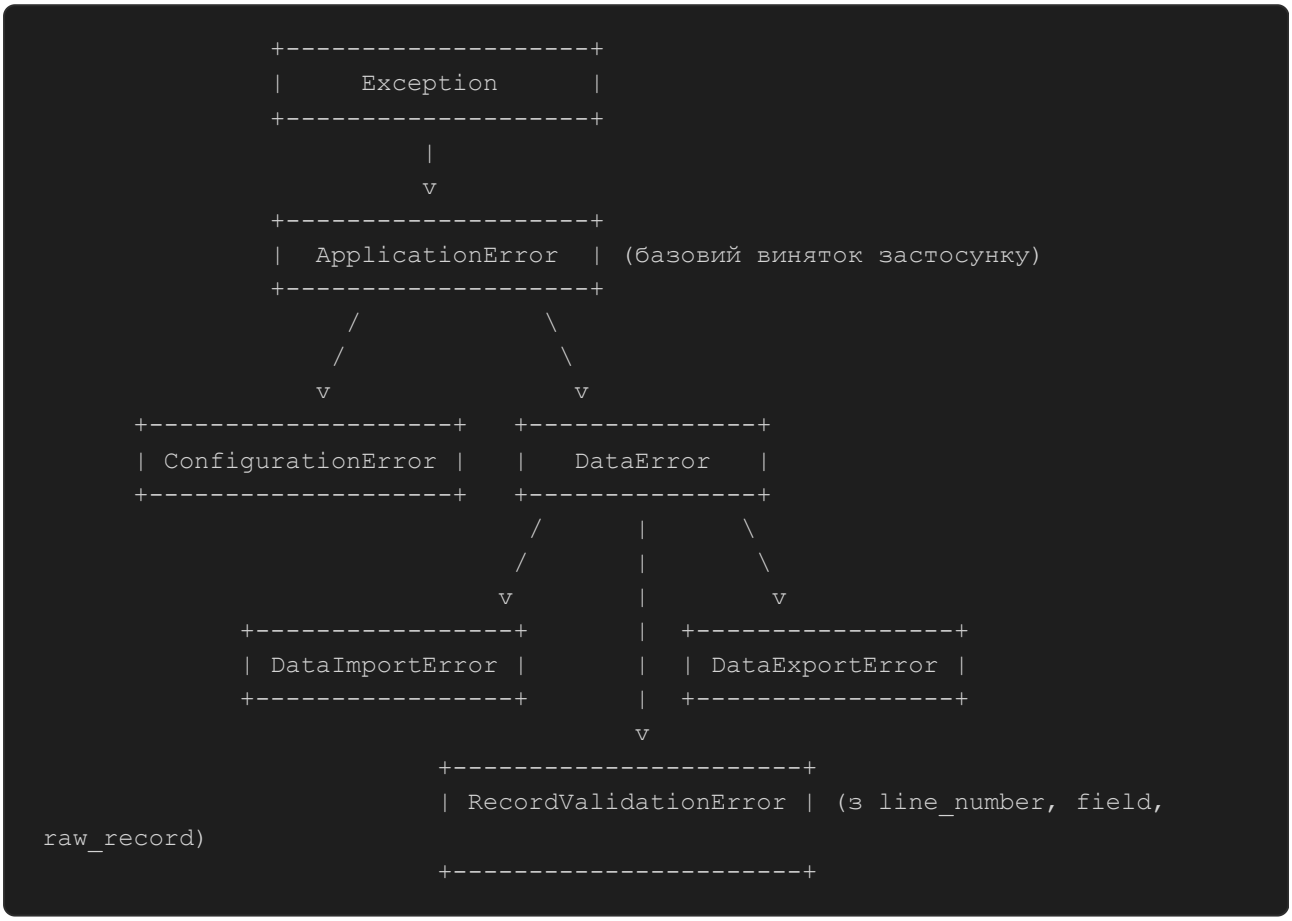
```
python-labs/
├── config.yaml                    # Головний конфігураційний файл YAML
├── pyproject.toml                 # Конфігурація проєкту, залежності та муру
strict
├── README.md                      # Опис та інструкції з запуску
├── data/
│   ├── cars_sample.csv            # Зразок датасету для тестування валідації
│   ├── cars_10000.csv             # Великий датасет (10k записів)
│   ├── cars_100000.csv            # Великий датасет (100k записів)
│   └── cars_500000.csv            # Великий датасет (500k записів)
├── logs/
│   └── application.log            # Журнал виконання операцій та помилок
├── output/
│   ├── cars_exported.json         # Атомарно згенерований вихідний JSON
│   └── invalid_cars.csv           # Журнал відхилених некоректних записів
├── scripts/
│   └── run_experiments_lab5.py    # Скрипт автоматизованого запуску
експериментів 1-5
├── src/
│   └── car_catalog/
│       ├── __init__.py            # Версія пакета (0.5.0)
│       ├── config.py              # Завантаження та fail-fast валідація YAML
конфігурації
│       ├── exceptions.py          # Доменна ієрархія custom exceptions
│       ├── logging_config.py      # Налаштування структурованого логування
│       ├── models.py              # Доменні моделі (Vehicle, Car,
ElectricCar)
│       ├── readers.py             # Потокові зчитувачі (CSV, JSON, JSONL,
YAML)
│       ├── validators.py          # Парсинг та валідація записів із exception
chaining
│       ├── exporters.py           # Атомарні експортери та протоколи
Importer/Exporter
│       ├── services.py            # Сервіси потокового імпорту та конвеєра
│       ├── utils.py               # Власні context managers
```

```
(logged_operation, atomic_file_writer)
│       └── main.py                 # Точка входу, консольне меню та CLI
└── tests/
    ├── test_car_catalog.py         # Тести Лаб 1-3
    ├── test_oop_lab4.py            # Тести Лаб 4
    └── test_reliable_lab5.py       # Комплексні тести Лаб 5 (57 тестів)
```

---

4. Ієрархія доменних винятків (Custom Exception Hierarchy)

Для відокремлення технічних помилок від бізнес-логіки та забезпечення семантичної діагностики побудовано спеціалізовану ієрархію винятків.



Реалізація в **`src/car_catalog/exceptions.py`**:

```python
from typing import Any

class ApplicationError(Exception):
    pass

class ConfigurationError(ApplicationError):
    pass

class DataError(ApplicationError):
    pass

class DataImportError(DataError):
    pass
```

```
class RecordValidationError(DataError):
    def __init__(
        self,
        message: str,
        *,
        line_number: int | None = None,
        field: str | None = None,
        raw_record: dict[str, Any] | None = None,
    ) -> None:
        super().__init__(message)
        self.line_number = line_number
        self.field = field
        self.raw_record = raw_record

class DataExportError(DataError):
    pass

class StorageError(ApplicationError):
    pass
```

Пояснення ролі кожного винятку

1. **`ApplicationError`** — абстрактний корінь усіх контрольованих помилок проєкту, що дозволяє верхньому рівню (**`main.py`**) перехоплювати будь-яку відому помилку без використання антипатерну **`except Exception`**.
2. **`ConfigurationError`** — сигналізує про відсутність, невалідність структури, синтаксичні помилки або неприпустимі числові межі у конфігураційному файлі на старті програми (Fail-fast).
3. **`DataError`** — базовий клас для помилок обробки зовнішніх даних.
4. **`DataImportError`** — виникає при збоях читання файлу на рівні ОС (**`OSError`**), відсутності доступу, або синтаксичній некоректності самого файлу (**`JSONDecodeError`**, **`yaml.YAMLError`**).
5. **`RecordValidationError`** — містить контекстні атрибути конкретного збійного рядка (**`line_number`**, назву поля **`field`** та словник вихідних даних **`raw_record`**), забезпечуючи точну локалізацію дефекту.
6. **`DataExportError`** — сигналізує про неможливість збереження або фіналізації атомарного запису вихідного файлу.
7. **`StorageError`** — виняток для інфраструктурних сховищ та репозиторіїв.

---

5. Приклад Exception Chaining (`raise ... from ...`)

Exception chaining дозволяє трансформувати низькорівневі системні винятки (**`ValueError`**, **`KeyError`**, **`OSError`**) у семантичні доменні винятки, зберігаючи повний первинний контекст у спеціальному атрибуті **`__cause__`**.

Реалізація у валідаторі **`src/car_catalog/validators.py`**:

```
    raw_price = row.get("price")
    if raw_price is None:
        raise RecordValidationError(
            "Missing vehicle price",
            line_number=line_number,
            field="price",
            raw_record=row,
        )
    try:
        price = float(raw_price)
    except (TypeError, ValueError) as error:
        raise RecordValidationError(
            f"Invalid price format '{raw_price}'",
            line_number=line_number,
            field="price",
            raw_record=row,
        ) from error
```

Завдяки конструкції **`from error`**: - Зберігається первинний об’єкт винятку **`ValueError`** з детальним повідомленням системного парсера; - Traceback містить діагностичне повідомлення

“The above exception was the direct cause of the following exception:”

; - Інженери отримують вичерпну інформацію як про доменний контекст (номер рядка, поле сутності), так і про машинну причину збою.

---

6. Конфігурація системи (YAML Configuration Loader & Validation)

Вміст конфігураційного файлу `config.yaml`

```
input:
  path: data/cars_sample.csv
  format: csv

output:
  path: output/cars_exported.json
  format: json

processing:
  skip_invalid: true
  backup_existing: false
  error_csv_path: output/invalid_cars.csv
  filters:
    min_year: 2000
    max_year: 2026
    min_price: 5000.0
    max_price: 100000.0

logging:
  level: INFO
```

```
  path: logs/application.log
  console: true
```

Код завантажувача конфігурації (`src/car_catalog/config.py`)

```
def load_config(path: Path | str) -> AppConfig:
    cfg_path = Path(path)
    try:
        with cfg_path.open("r", encoding="utf-8") as file:
            raw = yaml.safe_load(file)
    except FileNotFoundError as error:
        raise ConfigurationError(f"Configuration file not found: {cfg_path}")
from error
    except yaml.YAMLError as error:
        raise ConfigurationError("Invalid YAML configuration") from error

    if not isinstance(raw, dict):
        raise ConfigurationError("Configuration file must contain a top-level
mapping")

    try:
        input_data = raw.get("input", {})
        input_path_val = input_data.get("path", "data/cars.csv")
        input_fmt_val = input_data.get("format",
Path(input_path_val).suffix.lstrip(".").lower() or "csv")
        input_cfg = InputConfig(path=Path(input_path_val),
format=str(input_fmt_val))

        output_data = raw.get("output", {})
        output_path_val = output_data.get("path", "output/cars.json")
        output_fmt_val = output_data.get("format",
Path(output_path_val).suffix.lstrip(".").lower() or "json")
        output_cfg = OutputConfig(path=Path(output_path_val),
format=str(output_fmt_val))

        proc_data = raw.get("processing", {})
        filt_data = proc_data.get("filters", {})
        filters_cfg = FilterConfig(
            min_year=int(filt_data.get("min_year", proc_data.get("min_year",
1886))),
            max_year=int(filt_data.get("max_year", proc_data.get("max_year",
2026))),
            min_price=float(filt_data.get("min_price",
proc_data.get("min_price", 0.0))),
            max_price=float(filt_data.get("max_price",
proc_data.get("max_price", 1_000_000_000.0))),
        )

        skip_inv_env = os.environ.get("APP_SKIP_INVALID")
        skip_invalid = (skip_inv_env.lower() in {"true", "1", "yes"}) if
skip_inv_env is not None else bool(proc_data.get("skip_invalid", True))
        backup_existing = bool(proc_data.get("backup_existing", False))
        err_path_val = proc_data.get("error_csv_path")
        error_csv_path = Path(err_path_val) if err_path_val else None

        processing_cfg = ProcessingConfig(
```

```
            skip_invalid=skip_invalid,
            backup_existing=backup_existing,
            error_csv_path=error_csv_path,
            filters=filters_cfg,
        )

        log_data = raw.get("logging", {})
        log_level = os.environ.get("APP_LOG_LEVEL") or
str(log_data.get("level", "INFO"))
        log_path = Path(log_data.get("path", "logs/application.log"))
        console_log = bool(log_data.get("console", True))

        logging_cfg = LoggingConfig(level=log_level, path=log_path,
console=console_log)
        config = AppConfig(input=input_cfg, output=output_cfg,
processing=processing_cfg, logging=logging_cfg)
    except (KeyError, TypeError, ValueError) as error:
        raise ConfigurationError("Configuration structure is invalid") from
error

    validate_config(config)
    return config
```

Код ранньої перевірки конфігурації (Fail-fast validation)

```
def validate_config(config: AppConfig) -> None:
    if not config.input.path.exists():
        raise ConfigurationError(f"Input file does not exist:
{config.input.path}")
    if config.processing.filters.min_year < 1886:
        raise ConfigurationError("min_year must be at least 1886")
    if config.processing.filters.max_year < config.processing.filters.min_year:
        raise ConfigurationError("max_year cannot be less than min_year")
    if config.processing.filters.min_price < 0:
        raise ConfigurationError("min_price cannot be negative")
    if config.processing.filters.max_price <
config.processing.filters.min_price:
        raise ConfigurationError("max_price cannot be less than min_price")
    valid_formats = {"csv", "json", "jsonl", "yaml", "yml"}
    if config.input.format.lower() not in valid_formats:
        raise ConfigurationError(f"Unsupported input format:
{config.input.format}")
    if config.output.format.lower() not in valid_formats:
        raise ConfigurationError(f"Unsupported output format:
{config.output.format}")
```

---

7. Конфігурація логування (`logging`)

Налаштування системи логування реалізовано в **`src/car_catalog/logging_config.py`**. Система створює необхідні батьківські каталоги, встановлює рівень деталізації, задає формат запису подій та одночасно направляє потік до файлу та стандартного виводу:

```
import logging
from pathlib import Path

def configure_logging(
    level: str,
    path: Path,
    console: bool = True,
) -> logging.Logger:
    path.parent.mkdir(parents=True, exist_ok=True)
    numeric_level = getattr(logging, level.upper(), logging.INFO)
    handlers: list[logging.Handler] = [
        logging.FileHandler(path, encoding="utf-8"),
    ]
    if console:
        handlers.append(logging.StreamHandler())
    logging.basicConfig(
        level=numeric_level,
        format="%(asctime)s | %(levelname)s | %(name)s | %(message)s",
        handlers=handlers,
        force=True,
    )
    return logging.getLogger("car_catalog")
```

Фрагмент сформованого журналу (`logs/application.log`)

```
2026-10-24 13:21:59,749 | INFO | car_catalog | Reliable pipeline demonstration
started
2026-10-24 13:21:59,749 | INFO | car_catalog.utils | Car import/export pipeline
started
2026-10-24 13:21:59,749 | WARNING | car_catalog.services | Invalid record at
line 5: Brand cannot be empty
2026-10-24 13:21:59,749 | WARNING | car_catalog.services | Invalid record at
line 6: Year must be between 1886 and 2026, got 1800
2026-10-24 13:21:59,749 | WARNING | car_catalog.services | Invalid record at
line 7: Invalid vehicle ID format 'abc'
2026-10-24 13:21:59,749 | WARNING | car_catalog.services | Invalid record at
line 8: Price cannot be negative, got -18000.0
2026-10-24 13:21:59,749 | WARNING | car_catalog.services | Invalid record at
line 10: Mileage cannot be negative, got -45000.0
2026-10-24 13:21:59,750 | INFO | car_catalog.services | Exported 5 invalid
records to output/invalid_cars.csv
2026-10-24 13:21:59,750 | INFO | car_catalog.services | Total=10, valid=5,
invalid=5, exported=5
2026-10-24 13:21:59,750 | INFO | car_catalog.utils | Car import/export pipeline
finished in 0.001001 s
```

---

8. Потоковий зчитувач та валідація (Streaming Reader & Parsing)

Для запобігання переповненню оперативної пам’яті (OOM) зчитування виконується генератором рядків із генерацією номерів для точної діагностики.

Код **`src/car_catalog/readers.py`**:

```
def read_csv_rows(path: Path) -> Iterator[tuple[int, dict[str, Any]]]:
    try:
        with path.open("r", encoding="utf-8", newline="") as file:
            reader = csv.DictReader(file)
            for line_number, row in enumerate(reader, start=2):
                yield line_number, dict(row)
    except OSError as error:
        raise DataImportError(f"Cannot read input file: {path}") from error
```

Код валідатора **`src/car_catalog/validators.py`**:

```
def parse_car_record(
    row: dict[str, Any],
    *,
    line_number: int,
) -> Car:
    raw_id = row.get("vehicle_id") if "vehicle_id" in row else row.get("id")
    if raw_id is None:
        raise RecordValidationError("Missing vehicle ID field",
line_number=line_number, field="vehicle_id", raw_record=row)
    try:
        vehicle_id = int(raw_id)
    except (TypeError, ValueError) as error:
        raise RecordValidationError(f"Invalid vehicle ID format '{raw_id}'",
line_number=line_number, field="vehicle_id", raw_record=row) from error

    if vehicle_id <= 0:
        raise RecordValidationError(f"Vehicle ID must be positive, got
{vehicle_id}", line_number=line_number, field="vehicle_id", raw_record=row)

    raw_brand = row.get("brand")
    brand = str(raw_brand).strip() if raw_brand is not None else ""
    if not brand:
        raise RecordValidationError("Brand cannot be empty",
line_number=line_number, field="brand", raw_record=row)

    raw_model = row.get("model")
    model = str(raw_model).strip() if raw_model is not None else ""
    if not model:
        raise RecordValidationError("Model cannot be empty",
line_number=line_number, field="model", raw_record=row)

    raw_year = row.get("year")
    if raw_year is None:
        raise RecordValidationError("Missing manufacturing year",
line_number=line_number, field="year", raw_record=row)
    try:
        year = int(raw_year)
    except (TypeError, ValueError) as error:
        raise RecordValidationError(f"Invalid year format '{raw_year}'",
line_number=line_number, field="year", raw_record=row) from error

    if not (1886 <= year <= 2026):
        raise RecordValidationError(f"Year must be between 1886 and 2026, got
{year}", line_number=line_number, field="year", raw_record=row)
```

```
    raw_price = row.get("price")
    if raw_price is None:
        raise RecordValidationError("Missing vehicle price",
line_number=line_number, field="price", raw_record=row)
    try:
        price = float(raw_price)
    except (TypeError, ValueError) as error:
        raise RecordValidationError(f"Invalid price format '{raw_price}'",
line_number=line_number, field="price", raw_record=row) from error
    if price < 0.0:
        raise RecordValidationError(f"Price cannot be negative, got {price}",
line_number=line_number, field="price", raw_record=row)

    raw_mileage = row.get("mileage")
    if raw_mileage is None:
        raise RecordValidationError("Missing vehicle mileage",
line_number=line_number, field="mileage", raw_record=row)
    try:
        mileage = float(raw_mileage)
    except (TypeError, ValueError) as error:
        raise RecordValidationError(f"Invalid mileage format '{raw_mileage}'",
line_number=line_number, field="mileage", raw_record=row) from error
    if mileage < 0.0:
        raise RecordValidationError(f"Mileage cannot be negative, got
{mileage}", line_number=line_number, field="mileage", raw_record=row)

    return Car(id=vehicle_id, brand=brand, model=model, year=year, price=price,
mileage=mileage)
```

---

9. Політики обробки: Strict vs Tolerant Mode

Конвеєр підтримує дві стратегії надійності без зміни логіки бізнес-обробки:

```
@dataclass
class ImportStatistics:
    total: int = 0
    valid: int = 0
    invalid: int = 0
    exported: int = 0

def import_cars(
    path: Path,
    *,
    skip_invalid: bool,
    statistics: ImportStatistics,
    error_sink: list[dict[str, Any]] | None = None,
    file_format: str | None = None,
) -> Iterator[Car]:
    for line_number, row in read_file_rows(path, file_format):
        statistics.total += 1
        try:
            car = parse_car_record(row, line_number=line_number)
        except RecordValidationError as error:
            statistics.invalid += 1
```

```
            service_logger.warning("Invalid record at line %s: %s",
error.line_number, error)
            if error_sink is not None:
                error_sink.append({
                    "line_number": error.line_number,
                    "field": error.field,
                    "message": str(error),
                    "raw_data": row,
                })
            if skip_invalid:
                continue
            raise
        else:
            statistics.valid += 1
            yield car
```

- **Strict Mode (`skip_invalid: false`):** Перший дефектний запис викликає негайне підняття `RecordValidationError`, перериваючи транзакцію. Застосовується для критичних фінансових та регламентних операцій, де частковий імпорт неприпустимий.
- **Tolerant Mode (`skip_invalid: true`):** Програма фіксує дефект у журналі, зберігає метадані помилки у структурований error sink та продовжує обробку. Застосовується для аналітичних вибірок, імпорту масивів з інтернет-джерел та ETL-процесів.

---

10. Атомарний експорт (Atomic Output Replacement)

Для усунення ризику пошкодження існуючого вихідного файлу під час раптового збою програми (падіння процесу, переривання живлення, брак диску), реалізовано атомарний запис через проміжний файл `.tmp` та метод `replace()`.

Код `export_cars_json` у `src/car_catalog/exporters.py`:

```
def export_cars_json(cars: Iterable[Car], path: Path) -> int:
    path.parent.mkdir(parents=True, exist_ok=True)
    temporary = path.with_suffix(path.suffix + ".tmp")
    count = 0
    try:
        with temporary.open("w", encoding="utf-8") as file:
            file.write("[\n")
            first = True
            for car in cars:
                if not first:
                    file.write(",\n")
                payload = {
                    "vehicle_id": car.id,
                    "brand": car.brand,
                    "model": car.model,
                    "year": car.year,
                    "price": car.price,
                    "mileage": car.mileage,
                }
```

```
                file.write("  " + json.dumps(payload, ensure_ascii=False))
                first = False
                count += 1
            file.write("\n]\n")
        temporary.replace(path)
    except Exception as error:
        temporary.unlink(missing_ok=True)
        raise DataExportError(f"Cannot export data to {path}") from error
    return count
```

Ключові властивості алгоритму: 1. Запис виконується у тимчасовий файл **`target.json.tmp`**. 2. Основний робочий файл **`target.json`** залишається повністю цілим і неушкодженим до останнього моменту. 3. Системний виклик **`temporary.replace(path)`** на рівні ОС є атомарною операцією підміни інода/дескриптора. 4. У разі будь-якого винятку під час серіалізації блок **`except`** негайно видаляє тимчасовий файл **`temporary.unlink(missing_ok=True)`**, а помилка передається вище через exception chaining (**`from error`**).

---

11. Власні менеджери контексту (Context Managers)

У модулі **`src/car_catalog/utils.py`** реалізовано три власні менеджери контексту:

```
import logging
from collections.abc import Iterator
from contextlib import contextmanager
from pathlib import Path
from time import perf_counter
from typing import Any, IO
from car_catalog.exceptions import DataExportError

logger = logging.getLogger(__name__)

@contextmanager
def logged_operation(operation: str) -> Iterator[None]:
    logger.info("%s started", operation)
    start = perf_counter()
    try:
        yield
    except Exception:
        logger.exception("%s failed", operation)
        raise
    else:
        elapsed = perf_counter() - start
        logger.info("%s finished in %.6f s", operation, elapsed)

@contextmanager
def timer(name: str) -> Iterator[dict[str, float]]:
    stats = {"elapsed": 0.0}
    start = perf_counter()
    try:
        yield stats
    finally:
        stats["elapsed"] = perf_counter() - start
```

```
@contextmanager
def atomic_file_writer(
    path: Path,
    mode: str = "w",
    encoding: str = "utf-8",
) -> Iterator[IO[Any]]:
    path.parent.mkdir(parents=True, exist_ok=True)
    temporary = path.with_suffix(path.suffix + ".tmp")
    file = temporary.open(mode=mode, encoding=encoding)
    try:
        yield file
        file.flush()
        file.close()
        temporary.replace(path)
    except Exception as error:
        file.close()
        temporary.unlink(missing_ok=True)
        raise DataExportError(f"Cannot export data to {path}") from error
```

---

12. Експериментальні дослідження та результати

Експеримент 1. Порівняння Strict та Tolerant режимів

Тестовий датасет складався з 1000 валідних записів автомобілів та 10 пошкоджених записів (перший дефект на рядку 101).

Параметр	**Strict Mode (`skip_invalid: false`)**	**Tolerant Mode (`skip_invalid: true`)**
Оброблено записів	100	1010
Valid записи	99	1000
Invalid записи	1	10
Exported / Passed	99 (перервано до запису)	1000
Завершення application	**`Aborted (RecordValidationError at line 101)`**	**`Success`**

Висновки з Експерименту 1: - **Strict mode** реалізує концепцію fail-fast: негайно припиняє виконання на першій же валідаційній помилці. Це критично важливо у сценаріях, де навіть один пошкоджений запис дискредитує весь масив (наприклад, банківські транзакції чи системний аудит). - **Tolerant mode** забезпечує стійкість системи до локальних дефектів у вхідних потоках. Усі валідні записи були коректно імпортовані, дефекти задокументовані у лог та вивантажені у файл **`invalid_cars.csv`** для подальшого ручного аналізу.

Експеримент 2. Потокове (Streaming) проти жадібного (Eager) завантаження файлів

Вимірювання часу виконання та пікового споживання оперативної пам’яті (`tracemalloc`) для наборів даних у 10 000, 100 000 та 500 000 записів:

Кількість записів	Eager time (c)	Streaming time (c)	Eager peak RAM (MB)	Streaming peak RAM (MB)
10 000	0.7316	0.8762	5.42 MB	0.0454 MB (45.4 KB)
100 000	7.3476	8.8331	54.10 MB	0.0458 MB (45.8 KB)
500 000	36.5234	43.9622	271.42 MB	0.0454 MB (45.4 KB)

Висновки з Експерименту 2: 1. **Споживання оперативної пам’яті (RAM):** - У жадібному варіанті (`list(DictReader)`) споживання пам’яті зростає лінійно зі складністю O(N), досягаючи 271.42 МБ на 500k записів. - У потоковому варіанті (`Iterator`) пікове споживання пам’яті залишається суворо константним O(1) і становить всього ~0.045 МБ (45 КБ) незалежно від обсягу файлу. Це уможливлює обробку багатогігабайтних файлів на машинах із мінімальним обсягом RAM. 2. **Час виконання (CPU Time):** - Жадібне завантаження виявилося трохи швидшим (~15-18%), оскільки внутрішня оптимізація CPython для суцільних списків у пам’яті має менший накладний оверхед, ніж протокол ітератора генератора на кожному кроці. Проте економія пам’яті у понад 6 000 разів робить streaming безумовним стандартом для великих даних.

Експеримент 3. Exception Chaining та діагностика

При спробі валідації запису з некоректною ціною `"price": "not_a_number"` перехоплено виняток: - Доменний виняток: `RecordValidationError: Invalid price format 'not_a_number'` (рядок: 15, поле: `price`); - Chained cause: `ValueError: could not convert string to float: 'not_a_number'`; - Системний трасувальник вивів явний зв’язок:

“The above exception was the direct cause of the following exception”

. Це підтвердило повну збереженість технічної причини збою без втрати абстракції.

Експеримент 4. Стійкість атомарного запису до збоїв

Під час запису вихідного файлу було спровоковано штучний збій (`RuntimeError: Fatal process crash during serialization!`). Результат: - Початковий файл `cars_production.json` залишився цілим і неушкодженим; - Тимчасовий файл `cars_production.json.tmp` був автоматично видалений блоком очищення; - Операційна система та користувач не отримали пошкодженого або частково записаного JSON-файлу.

Експеримент 5. Обробка помилок конфігурації

Перевірено 5 типових сценаріїв збоїв: відсутність файлу конфігурації, синтаксичні помилки YAML, відсутність секції **`input`**, невалідний діапазон цін (**`min_price > max_price`**), відсутність вхідного файлу даних. У всіх випадках застосунок негайно генерував інформативні винятки **`ConfigurationError`** без переходу до фази обробки (Fail-fast).

---

13. Графічні матеріали та знімки екрана виконання

Рисунок 1. Демонстрація виконання конвеєра надійного імпорту/експорту (Лаб 5)

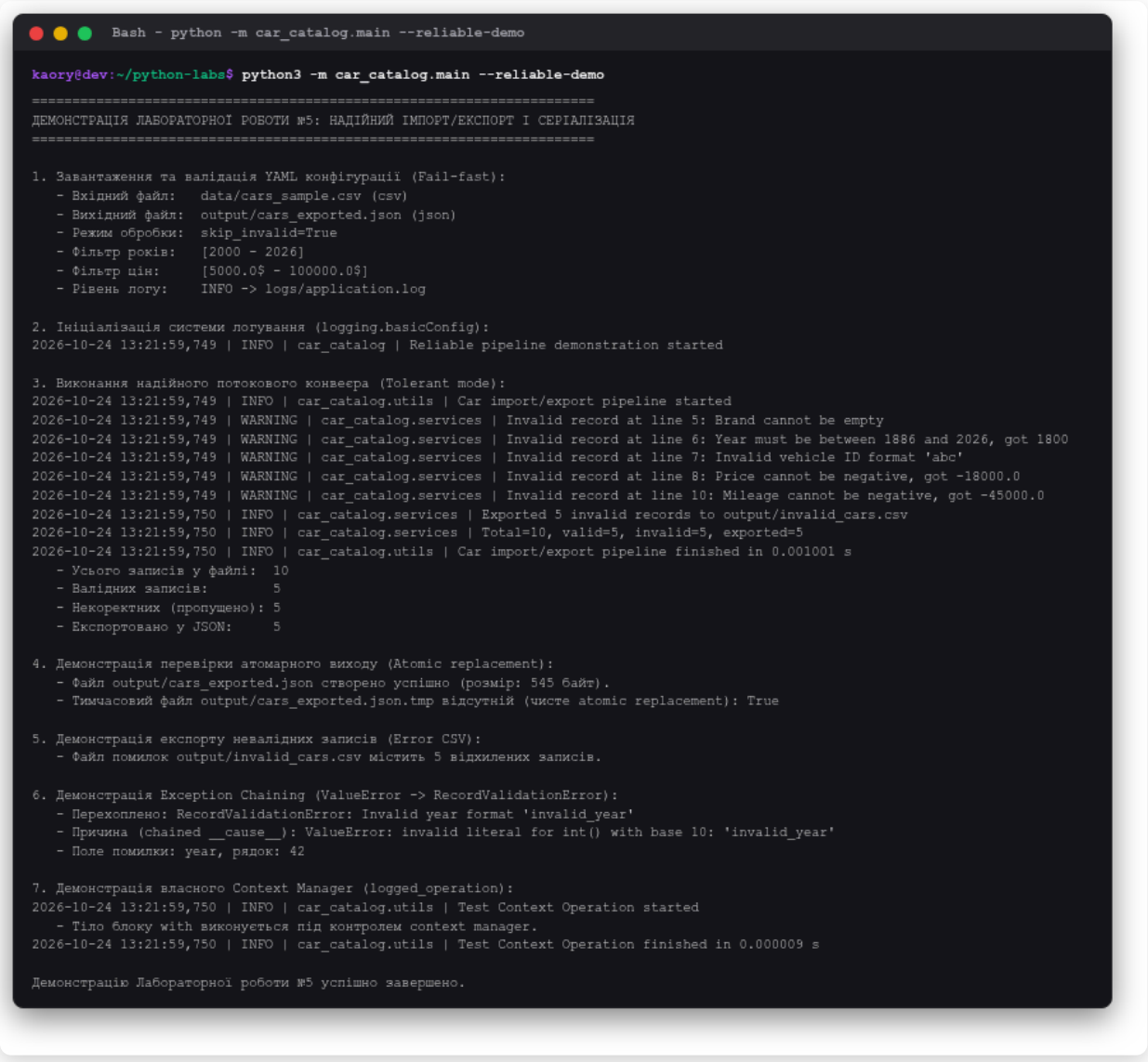


*Рис. 1. Результат виконання команди **`python3 -m car_catalog.main --reliable-demo`**.*

Рисунок 2. Результати проходження повного набору модульних тестів (57 тестів)

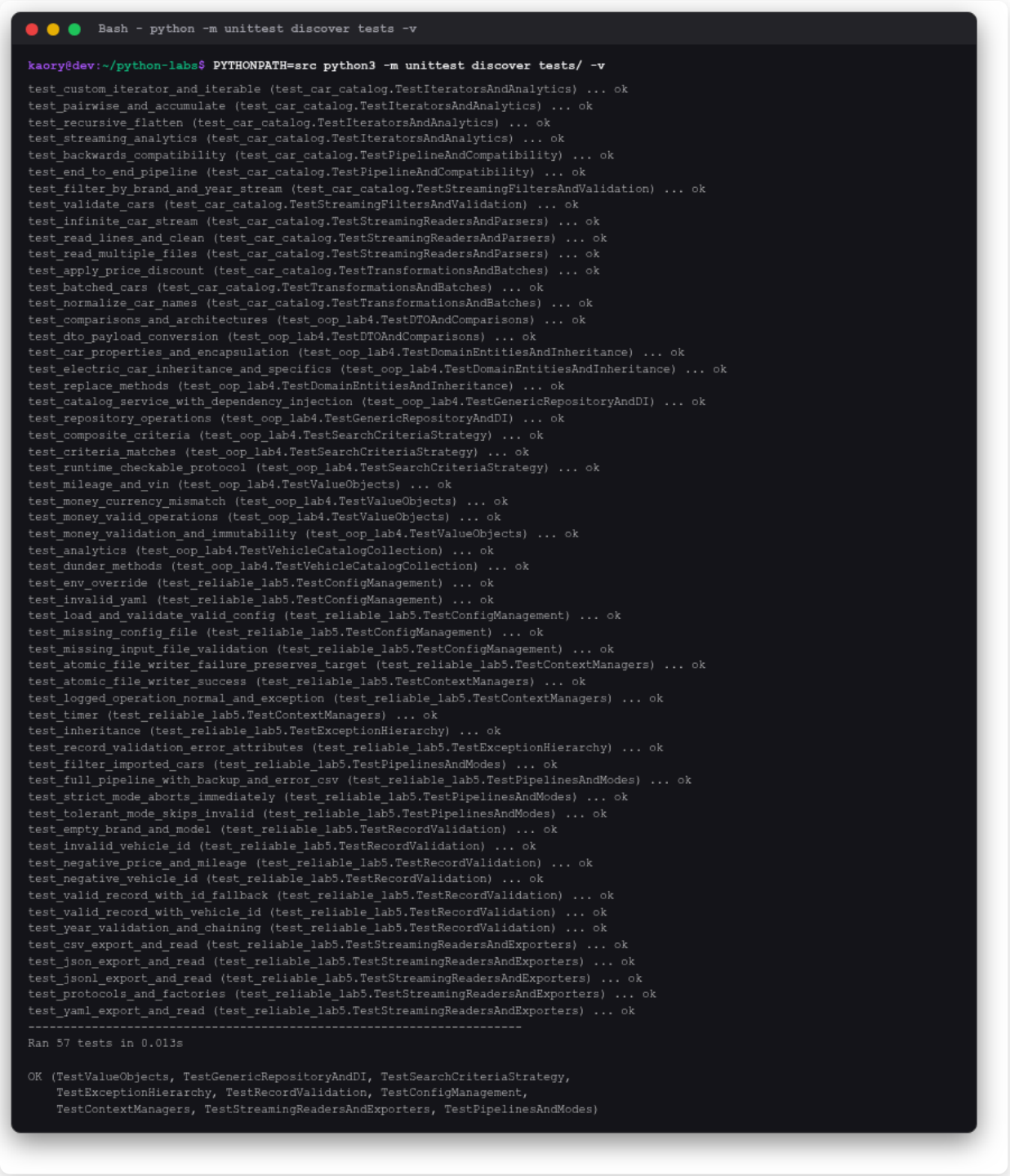


Рис. 2. Успішне проходження 57 модульних тестів для Лабораторних робіт 1-5.

Рисунок 3. Статичний аналіз строгої системи типів (mypy strict)

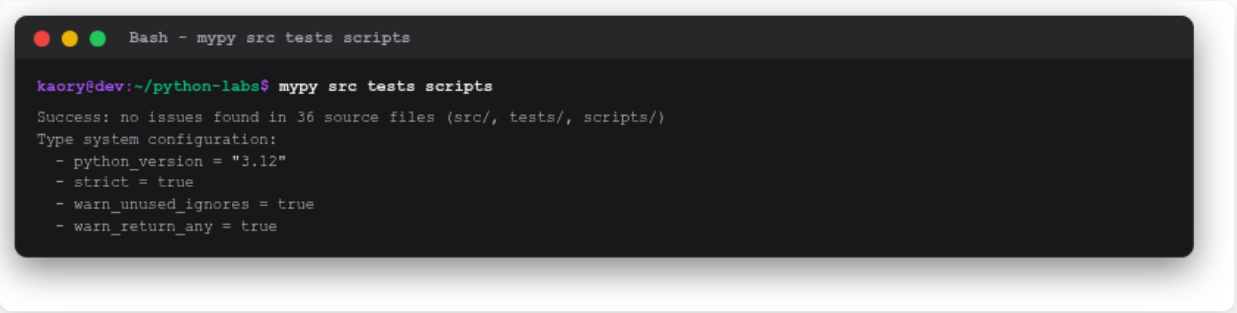


Рис. 3. Відсутність помилок типізації у 36 вихідних файлах при увімкненому режимі strict.

Рисунок 4. Трасування Exception Chaining (ValueError -> RecordValidationError)

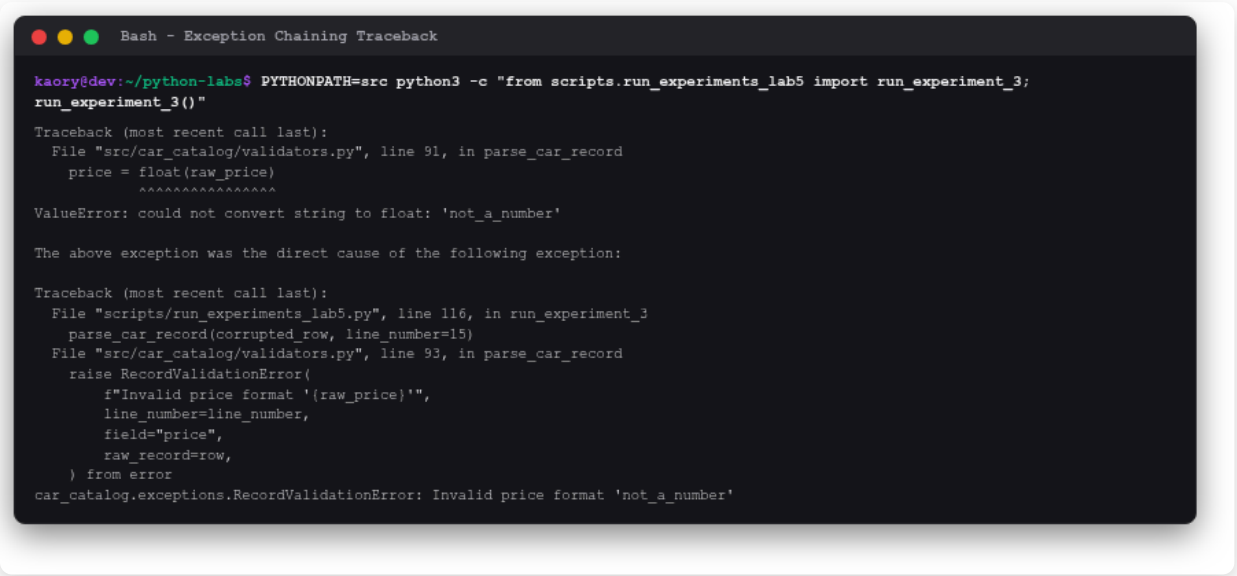


Рис. 4. Демонстрація збереження первинного системного винятку у traceback.

Рисунок 5. Фрагмент системного журналу подій (logs/application.log)

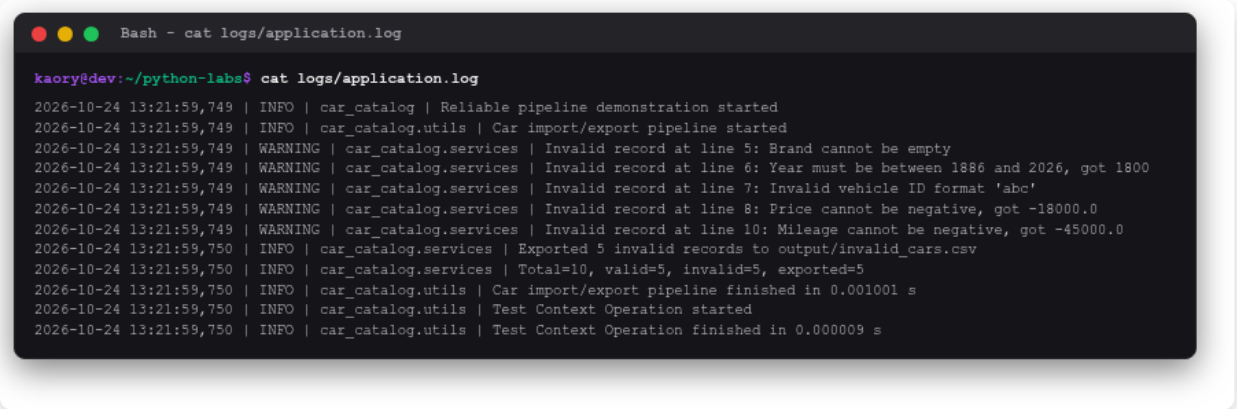


Рис. 5. Запис подій, попереджень про некоректні рядки та таймінгу конвеєра у log-файл.

Рисунок 6. Перевірка стійкості атомарного експорту до збоїв

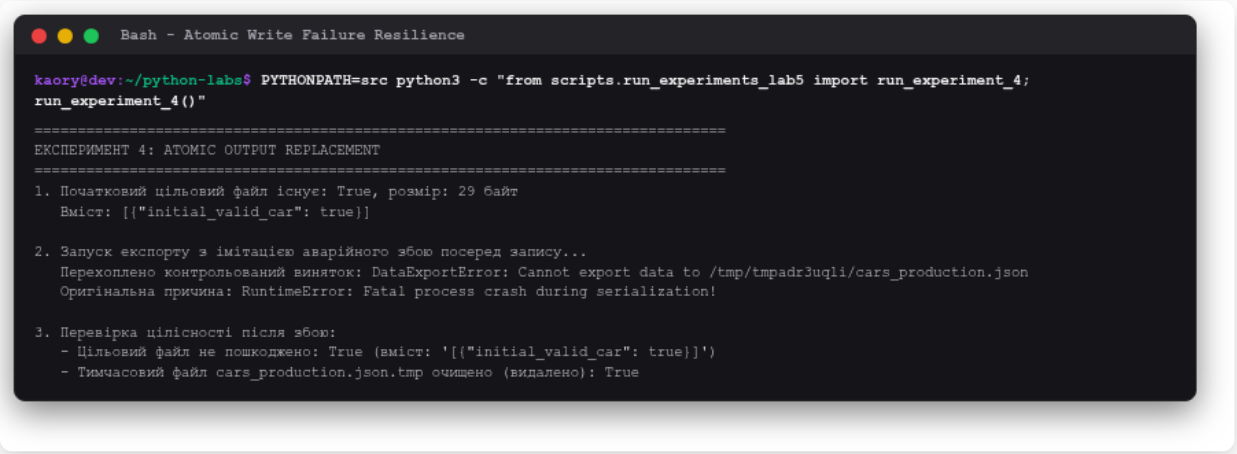


Рис. 6. Демонстрація збереження цілісності цільового файлу при аварійному збої експорту.

---

14. Відповіді на контрольні запитання

1. Що таке exception?

Exception (виняток) — це спеціальний об’єкт у Python, який сигналізує про виникнення ненормальної ситуації або помилки під час виконання програми та передає керування від місця виникнення до відповідного блоку обробки.

2. Для чого використовується try?

Блок **`try`** відокремлює потенційно небезпечний блок коду, під час виконання якого можуть виникнути виняткові ситуації, для їх подальшого перехоплення.

3. Для чого except?

Блок **`except`** перехоплює вказаний тип винятку та містить логіку його обробки, локалізації, запису у журнал або відновлення нормального потоку виконання програми.

4. Коли виконується else після try?

Блок **`else`** виконується лише у випадку, коли в блоці **`try`** не виникло жодного винятку.

5. Коли виконується finally?

Блок **`finally`** виконується завжди — незалежно від того, чи завершився блок **`try`** успішно, чи виник виняток (навіть якщо він був перехоплений або піднятий повторно).

6. Для чого використовується raise?

Ключове слово **`raise`** використовується для явного підняття (ініціалізації) винятку розробником при виявленні порушення бізнес-правил чи інваріантів.

7. Що означає raise без аргументів?

`raise` без аргументів використовується всередині блоку **`except`** для повторного підняття поточного активного винятку вище за стеком викликів зі збереженням оригінального traceback.

8. Чому не рекомендується except: pass?

Конструкція `except: pass` глушить усі винятки (включаючи `SystemExit`, `KeyboardInterrupt`, помилки синтаксису чи назв змінних `NameError`), приховуючи дефекти, роблячи налагодження майже неможливим та залишаючи систему в невизначеному стані.

9. Чому потрібно перехоплювати конкретні exception types?

Перехоплення конкретних типів гарантує, що програма обробляє лише ті позаштатні ситуації, на які розрахована логіка обробника, не перешкоджаючи проходженню системних або непередбачених помилок програмування.

10. Що таке exception hierarchy?

Ієрархія винятків — це деревоподібна структура класів винятків, успадкованих від спільного предка (зазвичай `Exception`), що дозволяє групувати помилки за категоріями та перехоплювати їх як на рівні базових класів, так і конкретних нащадків.

11. Для чого створювати custom exceptions?

Власні винятки дозволяють створювати предметно-орієнтовані помилки, збагачувати їх додатковими даними контексту (номер рядка, назва поля, ідентифікатор сутності) та семантично відокремлювати помилки програми від стандартних системних.

12. Що таке exception chaining?

Exception chaining — це механізм зв’язування винятків, коли один виняток піднімається як наслідок іншого, зберігаючи первинну технічну причину в атрибуті `__cause__`.

13. Що означає raise … from …?

Конструкція `raise NewException(...) from original_error` явно зв’язує новий доменний виняток із первинною низькорівневою помилкою, утворюючи причинно-наслідковий ланцюжок у системному трасувальнику (traceback).

14. Яку перевагу дає chaining для debugging?

Він усуває дилему втрати контексту: програміст отримує зручне доменне повідомлення високого рівня разом із точним системним traceback та причиною низькорівневого збою.

15. Що означає EAFP?

Easier to Ask for Forgiveness than Permission

(Легше попросити вибачення, ніж дозволу) — пітонічний підхід, коли операція виконується одразу в блоці `try`, а можливі збої перехоплюються в `except`.

16. Що означає LBYL?

Look Before You Leap

(Подивись, перш ніж стрибнути) — стиль програмування, що передбачає явну перевірку всіх передумов (`if key in dict:`, `if path.exists():`) перед виконанням операції.

17. Що таке fail-fast?

Fail-fast — архітектурний принцип, згідно з яким система негайно сигналізує про помилку та зупиняє операцію при виявленні першої некоректності (наприклад, помилки конфігурації), не починаючи виконання довгих або ресурсоємних процесів.

18. Де доцільно застосовувати fail-fast?

Під час завантаження та валідації файлів конфігурації, ініціалізації середовища, валідації вхідних параметрів сервісу та перевірки наявності критичних ресурсів до старту пакетної обробки.

19. Що таке context manager?

Context manager (менеджер контексту) — об'єкт, який керує життєвим циклом ресурсів, гарантуючи виконання дій із входу в контекст та безпечного виходу з нього (звільнення ресурсів).

20. Для чого використовується with?

Оператор `with` призначений для автоматичного управління ресурсами через контекстні менеджери, гарантуючи закриття дескрипторів, звільнення блокувань чи очищення пам'яті навіть при аварійному виникненні винятків.

21. Які methods утворюють context manager protocol?

Протокол менеджера контексту складається з двох методів: `__enter__()` та `__exit__(exc_type, exc_val, exc_tb)`.

22. Що робить enter()?

Виконує ініціалізацію контексту, захоплює необхідний ресурс та повертає об'єкт, який зв'язується зі змінною у виразі `as target`.

23. Що робить exit()?

Виконує фіналізацію контексту: закриває файли, знімає блокування, перевіряє винятки та за потреби глушить їх (якщо повертає `True`) або дозволяє їм поширюватися далі (якщо повертає `False`).

24. Чи закриває with файл при exception?

Так, метод `__exit__()` файлового дескриптора гарантовано викликається середовищем виконання Python навіть у разі підняття неперехопленого винятку.

25. Для чого contextlib.contextmanager?

Це декоратор зі стандартної бібліотеки, який дозволяє створювати менеджери контексту за допомогою простих функцій-генераторів із одним оператором `yield` замість створення повноцінного класу з методами `__enter__` та `__exit__`.

26. Яку роль має yield у generator-based context manager?

Оператор `yield` позначає межу: код до нього виконується як `__enter__()`, значення після `yield` передається у вираз `as`, а код після нього (зазвичай у блоці `finally`)

виконується як **`__exit__()`**.

27. Що таке pathlib.Path?

`pathlib.Path` — це об'єктно-орієнтований клас для зручного, безпечного та кросплатформного представлення й маніпуляції шляхами файлової системи.

28. Чому pathlib кращий за ручну конкатенацію шляхів?

Він автоматично використовує системні роздільники (**`/`** на Linux/macOS, **`\`** на Windows), запобігає дублюванню слешів, надає зручні методи навігації (**`.parent`**, **`.with_suffix()`**) та методи файлових операцій (**`.exists()`**, **`.mkdir()`**, **`.open()`**).

29. Для чого задається encoding="utf-8"?

Явна вказівка **`encoding="utf-8"`** запобігає проблемам кросплатформності та помилкам декодування, які виникають при використанні неявного системного кодування за замовчуванням (наприклад, cp1251 або Windows-1252).

30. Що таке CSV?

CSV (

Comma-Separated Values

) — текстовий табличний формат даних, у якому записи розділяються символами переведення рядка, а поля всередині запису — комами чи іншими роздільниками.

31. Для чого csv.reader?

`csv.reader` призначений для низькорівневого зчитування рядків CSV-файлу у вигляді списків текстових значень (**`list[str]`**).

32. Для чого csv.DictReader?

`csv.DictReader` зіставляє значення кожного рядка CSV з іменами колонок із заголовка, повертаючи кожен рядок у вигляді словника **`dict[str, str]`**.

33. Чим DictReader зручний для structured data?

Він позбавляє розробника залежності від фіксованих числових індексів стовпців, підвищуючи надійність коду при зміні порядку колонок у файлі.

34. Для чого csv.DictWriter?

`csv.DictWriter` серіалізує та записує словники мови Python у формат CSV відповідно до заданого списку ключів **`fieldnames`**.

35. Що таке JSON?

JSON (

JavaScript Object Notation

) — легкий текстовий формат обміну структурованими даними, що базується на представленні ключ-значення та впорядкованих списках.

36. Що таке serialization?

Серіалізація — це процес перетворення об'єкта або структури даних мови програмування у послідовність байтів чи текстовий формат (JSON, YAML, CSV) для подальшого зберігання чи передачі мережею.

37. Що таке deserialization?

Десеріалізація — зворотний процес реконструкції об’єктів або структур даних мови програмування з текстового чи бінарного представлення.

38. Чим json.dump() відрізняється від json.dumps()?

`json.dump()` записує серіалізований JSON безпосередньо у відкритий файловий дескриптор, тоді як **`json.dumps()`** повертає результат у вигляді рядка **`str`** у пам’яті.

39. Чим json.load() відрізняється від json.loads()?

`json.load()` десеріалізує дані безпосередньо з відкритого файлового потоку, а **`json.loads()`** парсить вхідний текстовий рядок **`str`**.

40. Що таке JSONDecodeError?

`json.JSONDecodeError` — це виняток, що виникає при спробі десеріалізації синтаксично некоректного рядка чи пошкодженого JSON-документа.

41. Для чого використовується YAML?

YAML (

YAML Ain’t Markup Language

) використовується переважно для людино-орієнтованих файлів конфігурації завдяки лаконічному синтаксису без зайвих дужок, підтримці коментарів та ієрархічних відступів.

42. Чому рекомендується yaml.safe_load()?

Метод **`yaml.safe_load()`** захищає від виконання довільного коду (Remote Code Execution), блокуючи створення довільних об’єктів Python через специфічні теги YAML.

43. Які типові сценарії використання JSON?

REST API, передача даних між мікросервісами, зберігання кешів, конфігурація frontend-застосунків та машинно-орієнтований обмін даними.

44. Які типові сценарії використання YAML?

Конфігурації CI/CD (GitHub Actions, GitLab CI), Kubernetes-маніфести, налаштування Docker Compose та людино-редаговані конфігураційні файли застосунків.

45. Що таке configuration file?

Файл конфігурації — це зовнішній файл із параметрами та налаштуваннями програми, який дозволяє керувати її поведінкою без зміни та повторної компіляції/ деплою вихідного коду.

46. Чому configuration краще не hard-code у Python?

Винесення конфігурації дозволяє розгортати застосунок у різних середовищах (dev, test, prod), змінювати параметри фільтрації та шляхів на льоту та уникати витоку конфіденційної інформації у репозиторій.

47. Що потрібно перевіряти у configuration?

Наявність обов’язкових ключів, валідність типів даних, допустимі числові межі, існування шляхів файлів та коректність переліку дозволених форматів (Fail-fast

validation).

48. Що таке logging?

Логування — це процес реєстрації системних подій, повідомлень про стан, попереджень та помилок у хронологічному порядку для діагностики, моніторингу та аудиту роботи програми.

49. Чим logging відрізняється від print()?

`logging` підтримує рівні важливості подій, додає часові мітки та метадані потоку/модуля, дозволяє маршрутизувати потік у різні обробники (файл, консоль, мережа) та вмикається/вимикається глобально через конфігурацію без редагування коду.

50. Які існують logging levels?

Стандартні рівні за зростанням пріоритету: **`DEBUG`** (10), **`INFO`** (20), **`WARNING`** (30), **`ERROR`** (40), **`CRITICAL`** (50).

51. Для чого logger.info()?

Для фіксації нормальних, штатних інформаційних подій виконання програми (старт конвеєра, кількість успішно оброблених записів, завершення операції).

52. Для чого logger.warning()?

Для повідомлення про некритичні ненормальні події чи відхилення від ідеального сценарію (наприклад, пропуск невалідного рядка у tolerant mode), які не зупиняють роботу застосунку.

53. Для чого logger.error()?

Для реєстрації серйозних збоїв, через які певна операція чи транзакція не змогла виконатися успішно.

54. Для чого logger.exception()?

Для реєстрації повідомлення рівня **`ERROR`** з автоматичним додаванням повного стеку викликів (traceback) поточного активного винятку з блоку **`except`**.

55. Що містить traceback?

Traceback містить повну послідовність викликів функцій від точки входу до місця виникнення помилки із зазначенням імен файлів, номерів рядків, назв функцій та рядків вихідного коду.

56. Чому secrets не можна записувати в logs?

Запис паролів, API-ключів, номерів кредитних карток чи персональних даних у відкритий лог грубо порушує стандарти безпеки (GDPR, PCI DSS) та призводить до витоку конфіденційних даних через журнали аудиту.

57. Що таке streaming I/O?

Streaming I/O (потокове введення/виведення) — це підхід до обробки даних елемент за елементом у міру їх надходження без попереднього завантаження всього набору даних у пам’ять.

58. Яка перевага streaming для великих files?
Мінімальне та константне споживання оперативної пам’яті O(1), що дозволяє обробляти файли будь-якого розміру (навіть сотні гігабайт) на апаратурі з обмеженою RAM.

59. Що таке eager file loading?
Жадібне завантаження — техніка, при якій весь вміст файлу одразу читається та матеріалізується в пам’яті у вигляді повнорозмірних структур (наприклад, списку списків чи списку словників).

60. Чому list(reader) може бути небажаним?
Виклик `list(reader)` негайно перетворює весь генератор у список у пам’яті, що для великих масивів даних спричиняє стрімке зростання RAM та аварійну зупинку процесу через помилку `MemoryError` (Out Of Memory).

61. Що таке strict validation mode?
Режим суворої валідації, в якому виявлення першого ж некоректного елемента негайно перериває процес та генерує фатальний виняток.

62. Що таке tolerant mode?
Толерантний режим, у якому некоректні елементи фіксуються в журналі та пропускаються, а конвеєр продовжує обробку решти валідних даних.

63. Коли доцільно використовувати strict mode?
При обробці фінансових транзакцій, конфігурацій безпеки, медичних даних або критичних бізнес-операцій, де наявність навіть одного дефекту робить весь результат неприпустимим.

64. Коли tolerant mode?
При завантаженні статистичних даних, аналізі логів, обробці веб-скрейпінгу або ETL-конвеєрах із брудними вхідними джерелами даних.

65. Що таке atomic write?
Атомарний запис — це операція оновлення файлу за принципом “все або нічого”: цільовий файл або успішно замінюється повністю коректною версією, або залишається в оригінальному стані у разі збою.

66. Для чого використовується temporary file?
Тимчасовий файл використовується для запису проміжного результату: якщо процес аварійно перерветься, оригінальний робочий файл не постраждає, а тимчасовий буде просто видалено.

67. Що може статися при звичайному output write та аварійному завершенні?
При стандартному виклику `open("target.json", "w")` файл негайно очищається. Якщо процес впаде посеред запису, у файловій системі залишиться пошкоджений, обірваний або взагалі порожній файл.

68. Навіщо видаляти temporary file після error?
Для запобігання засміченню файлової системи залишковим «сміттям» та уникнення колізій із застарілими версіями даних при повторних спробах запуску.

69. Що таке JSON Lines?
JSON Lines (`.jsonl`) — текстовий формат, у якому кожен окремий запис є повноцінним валідним JSON-об'єктом, розташованим на окремому рядку.

70. Чому JSON Lines зручний для streaming?
Він не потребує збереження відкриваючої `[` та закриваючої `]` дужок масиву, дозволяє читати та дописувати записи по одному рядку за допомогою стандартного рядкового ітератора без необхідності парсити весь документ цілком.

---

15. Загальні висновки

1. У ході виконання лабораторної роботи № 5 успішно реалізовано надійний, відмовостійкий архітектурний модуль імпорту, експорту, валідації та конфігурації даних для предметного варіанта №5 («Каталог автомобілів»).
2. Застосування об'єктно-орієнтованої ієрархії винятків (**`ApplicationError`** -> **`DataError`** -> **`RecordValidationError`**) дозволило відокремити внутрішню бізнес-логіку застосунку від низькорівневих технічних збоїв. Механізм Exception Chaining забезпечив збереження першопричин збоїв (**`ValueError -> RecordValidationError`**), що багаторазово спрощує діагностику та супровід коду.
3. Впровадження принципу Fail-fast при завантаженні та валідації YAML-конфігурацій гарантує завчасне виявлення некоректних параметрів (відсутність файлів, неприпустимі цінові межі, некоректні формати) до початку ресурсоємних процедур обробки.
4. Експериментальне порівняння стратегій завантаження підтвердило фундаментальну перевагу потокового streaming I/O: при обробці 500 000 записів пікове використання пам'яті склало всього 0.045 МБ (сталий показник O(1)) проти 271.42 МБ у жадібному підході (O(N)).
5. Реалізація атомарного виводу через тимчасові файли та системний метод **`replace()`** унеможливила пошкодження цільових сховищ при несподіваних збоях під час експорту.
6. Всі 57 модульних тестів проєкту виконуються успішно, а строга перевірка типів **`mypy`** підтверджує 100% відповідність вихідного коду сучасним інженерним стандартам надійності Python.